Claude Code In Depth

By Peter Cosemans
Level: 300 (Advanced)

euricom
TECH TRIBES

Copyright © - Euricom 2026

Install & Setup

euricom
TECH TRIBES

Install

To install Claude Code, use one of the following methods:

```
# Install (MacOS, Linux & WSL)
curl -fsSL https://claude.ai/install.sh | bash

# Install Powerhell
irm https://claude.ai/install.ps1 | iex

# After installation
cd your-awesome-project
claude
```

Native installations automatically update in the background to keep you on the latest version.

Login (Claude Subscription)

On first use, it will prompt you to log in

```
# Install (MacOS, Linux & WSL)
curl -fsSL https://claude.ai/install.sh | bash

# Install Powerhell
irm https://claude.ai/install.ps1 | iex

# After installation
cd your-awesome-project
claude
```

Native installations automatically update in the background to keep you on the latest version.

Authenticate with Anthropic API key

Get your API key from the Anthropic Console

```
# MacOS & Linux
export ANTHROPIC_API_KEY="sk-your-api-key-here"

# Powershell
$env:ANTHROPIC_API_KEY="sk-your-api-key-here"
```

Set the API key permanently (Windows GUI)

- Press `Win + R`, type `sysdm.cpl`, and hit Enter → open System Properties.
- Go to Advanced → Environment Variables
- Set the variable above

Authenticate with Anthropic API key

Mark the onboarding as completed in `~/.claude.json`

```
{
  "installMethod": "native",
  "autoUpdates": true,
  "hasCompletedOnboarding": true
}
```

And startup

```
# After config
cd your-awesome-project
claude
```

Authenticate with Azure

```
# Azure resource
export CLAUDE_CODE_USE_FOUNDRY=1
export ANTHROPIC_FOUNDRY_BASE_URL=https://{resource}.services.ai.azure.com

# Specify model deployment names, matching the deployment names in Foundry
export ANTHROPIC_DEFAULT_SONNET_MODEL="claude-sonnet-4-6"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="claude-haiku-4-5"
export ANTHROPIC_DEFAULT_OPUS_MODEL="claude-opus-4-6"

# Usage
claude
```

Foundation

euricom
TECH TRIBES

Run from project root

When you first launch Claude Code, run it from your project's root directory. Claude works best when it can see your whole proiect structure.

```
➜  poc-claude-code (main) claude

    *
    |
   ▟▛▜▙      Claude Code v2.1.45
   ▜▜▟▛      Sonnet 4.6 · Claude Team
   ▝▘▝▘      ~/git/euricom/poc-claude-code

❯ Try "write a test for README.md"

  Model: Sonnet 4.6 | Ctx: 0 | ⎇ main | (+0,-0) | Block:...
```

Initial setup

Creates a CLAUDE.md file. Think of it as Claude's memory. /init is a starting point. Review it, refine it, make it yours.

```
/init
```

Configures Shift+Enter for multiline input in terminals that don't support it natively
Also required to enable Alt+T (toggle extended thinking).

```
/terminal-setup
```

Shows status of IDE integrations (VS Code, JetBrains) and manages connections

```
/ide
```

Keyboard hotkeys

Key	Description
`shift+tab`	Cycle accept-edit / plan-mode / normal
`esc`	Interrupts. If you see the thinking block going off-rails, stop it early. Don't wait for Claude to finish making a mistake
`esc + esc`	Clears the prompt input
`ctrl+o`	Toggle verbose output (see what claude is thinking)
`ctrl+z`	Suspend to terminal (resume with ‘`fg`’) **Mac Only**
`ctrl+s`	Stash prompt
`ctrl+g`	Open default editor (for long prompts & plans)

https://code.claude.com/docs/en/interactive-mode

Keyboard hotkeys

Key	Description
`ctrl+v`	Past image from clipboard, not the standard cmd+v (for macos)
`ctrl+r`	Search through prompts
`up/down`	Walk through your previous prompts
`option/alt+t`	Toggle thinking mode
`ctrl-b`	Run bash in background (bash commands, list & stop with /tasks)
`ctrl-d` `ctrl-c`	Exit Claude Code

https://code.claude.com/docs/en/interactive-mode

Keyboard hotkeys - Prompt Edit

Key	Description
`option+left/right`	Word left or right
`ctrl-a`	Begin of line
`ctrl+e`	End of line
`ctrl+w`	Delete word
`ctrl+u`	Delete line

https://code.claude.com/docs/en/interactive-mode

Prompt prefixes

prefix	Description
`/{command}`	Tag command or skill
`!{command}`	Launch bash command
`@{file/dir}`	File path mention

https://code.claude.com/docs/en/interactive-mode

Build in (basic) commands

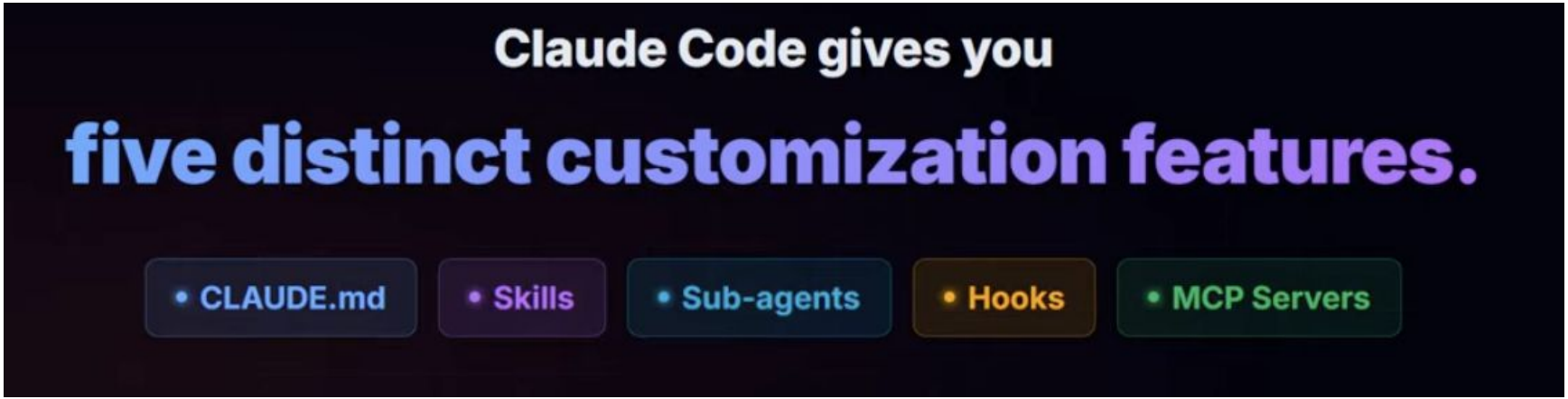
prefix	Description
`/clear`	Clear conversation history and free up context.
`/context`	Audit your context. Is it clean? Relevant?
`/models [model]`	Switching models.
`/doctor`	Check Claude Code installation health
`/usage`	Show plan usage limits and rate limit status
`/release-notes`	View the full changelog
`/btw`	Short question without interrupting claude
`...`	And many more

https://code.claude.com/docs/en/interactive-mode

Customization

euricom
TECH TRIBES

Customization



Get The Most Out of Your Claude Code Terminal

Claude.md

CLAUDE.md

A markdown file Claude reads at the start of every session.

ALWAYS ON No exceptions.

THINK OF IT AS

Your company handbook — everyone reads it, it always applies.

PUT YOUR NON-NEGOTIABLE STANDARDS HERE

CLAUDE.md

```
# CLAUDE.md — Project Standards

## Non-Negotiable Rules
- Always use TypeScript strict mode
- Use pnpm, never npm
- Never modify the database schema directly
```

- TypeScript strict mode — always
- Use pnpm, never npm
- Never modify the database schema

Get The Most Out of Your Claude Code Terminal

CLAUDE.md is hierarchical.

ENTERPRISE ~/.claude/enterprise.md
Applies to everyone in your org

USER ~/.claude.md
Follows you across all projects

PROJECT ./CLAUDE.md
Shared with your team via git

LOCAL ./CLAUDE.local.md
Just for you, this project only

More specific always wins.

It's a cascade — like CSS for your AI assistant.

Get The Most Out of Your Claude Code Terminal

Memory

Claude builds its own memory automatically.

As you work together, it notices patterns, preferences, and conventions. It writes them to a memory directory and loads them back next session.

.claude/memory/

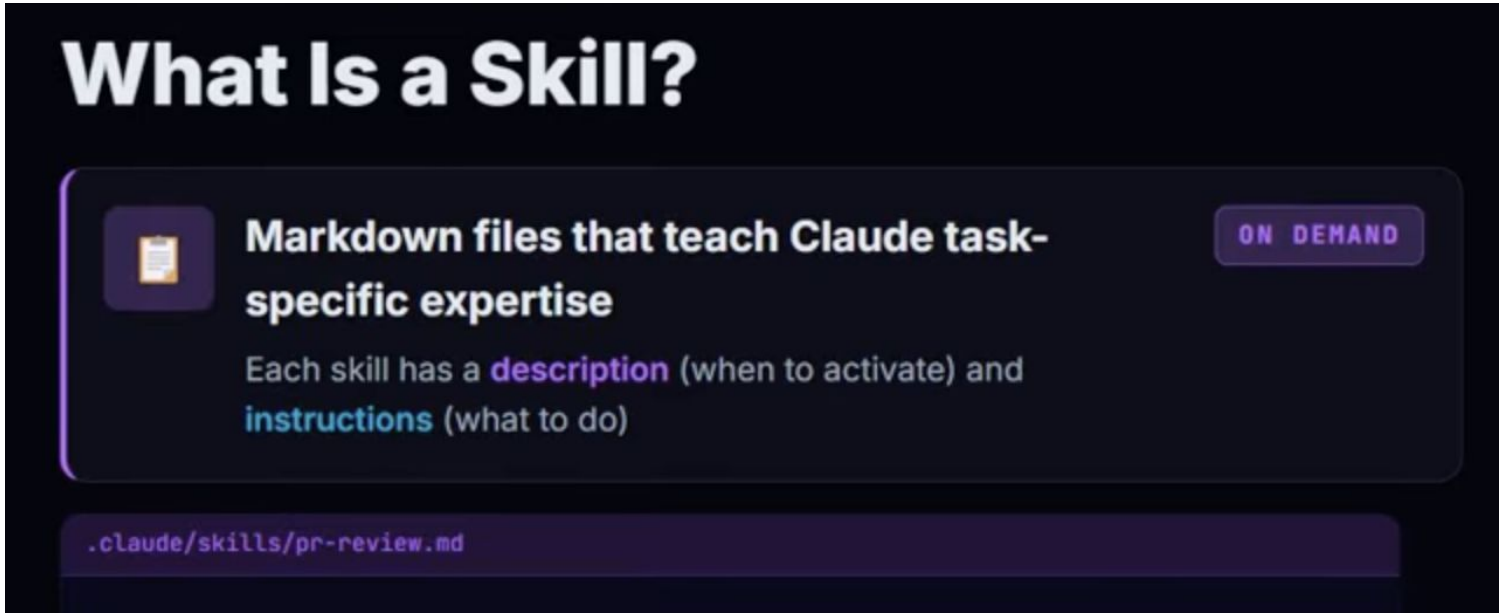
.claude/memory/preferences.md	AUTO-SAVED
.claude/memory/conventions.md	AUTO-SAVED
.claude/memory/patterns.md	AUTO-SAVED

DIY Smart C

```
/memory
```

Get The Most Out of Your Claude Code Terminal

Skill




```
/skills
```

Get The Most Out of Your Claude Code Terminal

Skill

Auto-Triggering Magic

Claude reads your request and matches against all skill descriptions

you type
"review this PR"

Claude matches
description keywords

loads skill
pr-review.md ✓

"review this PR"
→ Loads pr-review skill automatically
pr-review.md loaded ✓

"deploy to production"
→ Loads deployment skill automatically
deploy-checklist.md loaded ✓

💡 Only the description stays in context at all times — the full skill content loads on demand

Get The Most Out of Your Claude Code Terminal

Skill & Slash Commands

Slash Commands & Supporting Files

Skills and slash commands are the same thing — and skills can include extra files

You can invoke skills explicitly:

```
/review
/deploy
/commit
```

KEY INSIGHT

Anthropic **merged slash commands and skills** — they're the same thing now. Both create a slash command you can invoke directly.

Skills can include supporting files:

```
.claude/skills/
├── pr-review.md
├── pr-template.md
├── deploy.md
├── deploy-checklist.md
└── commit.md
└── commit-conventions.md
```

Templates, reference docs, scripts — Claude reads them only when the skill activates

Where you store it = who uses it:

```
~/claude/skills/ → personal, follows you everywhere
.claude/skills/ → shared with team via git
```

Sub-Agent

What Is a Sub-agent?

AI assistants running in their own context window

ISOLATED CONTEXT

Completely **isolated** from your main conversation — they do the heavy lifting so your context stays clean

Main Context
your conversation

delegates →
← summary only

Sub-agent Context
isolated window
reads dozens of files...

All that verbose output stays in the sub-agent's context. **Only the relevant summary comes back to you.**

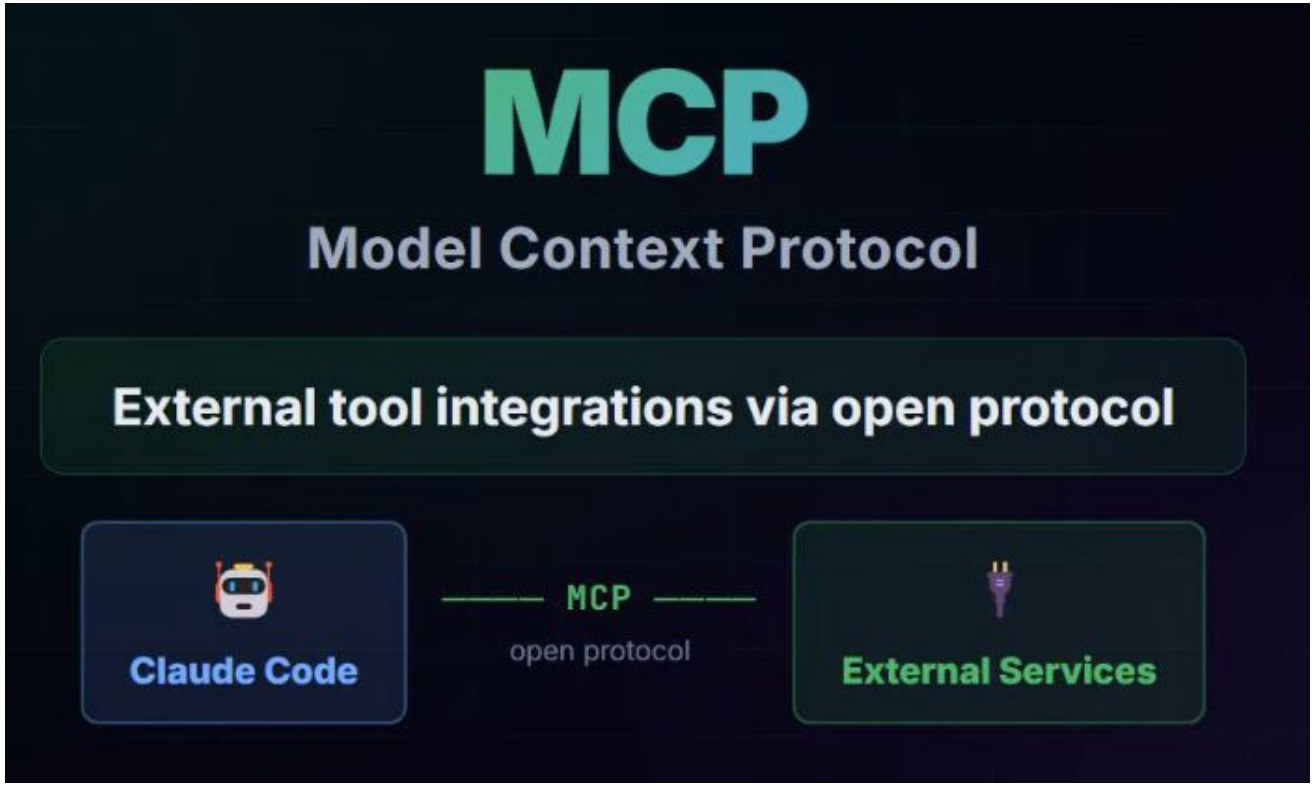
```
/agents
```

Hooks

What Is a Hook?
Shell commands that execute on lifecycle events
EVENT DRIVEN
Automatically
Deterministically
No LLM guessing
LIFECYCLE EVENT SEQUENCE
Session Start
Pre ToolUse
Post ToolUse
Stop
PreToolUse
PostToolUse
SessionStart
Stop

```
/hooks
```

MCP




```
/mcp
```

Get The Most Out of Your Claude Code Terminal

Key Rules for customization

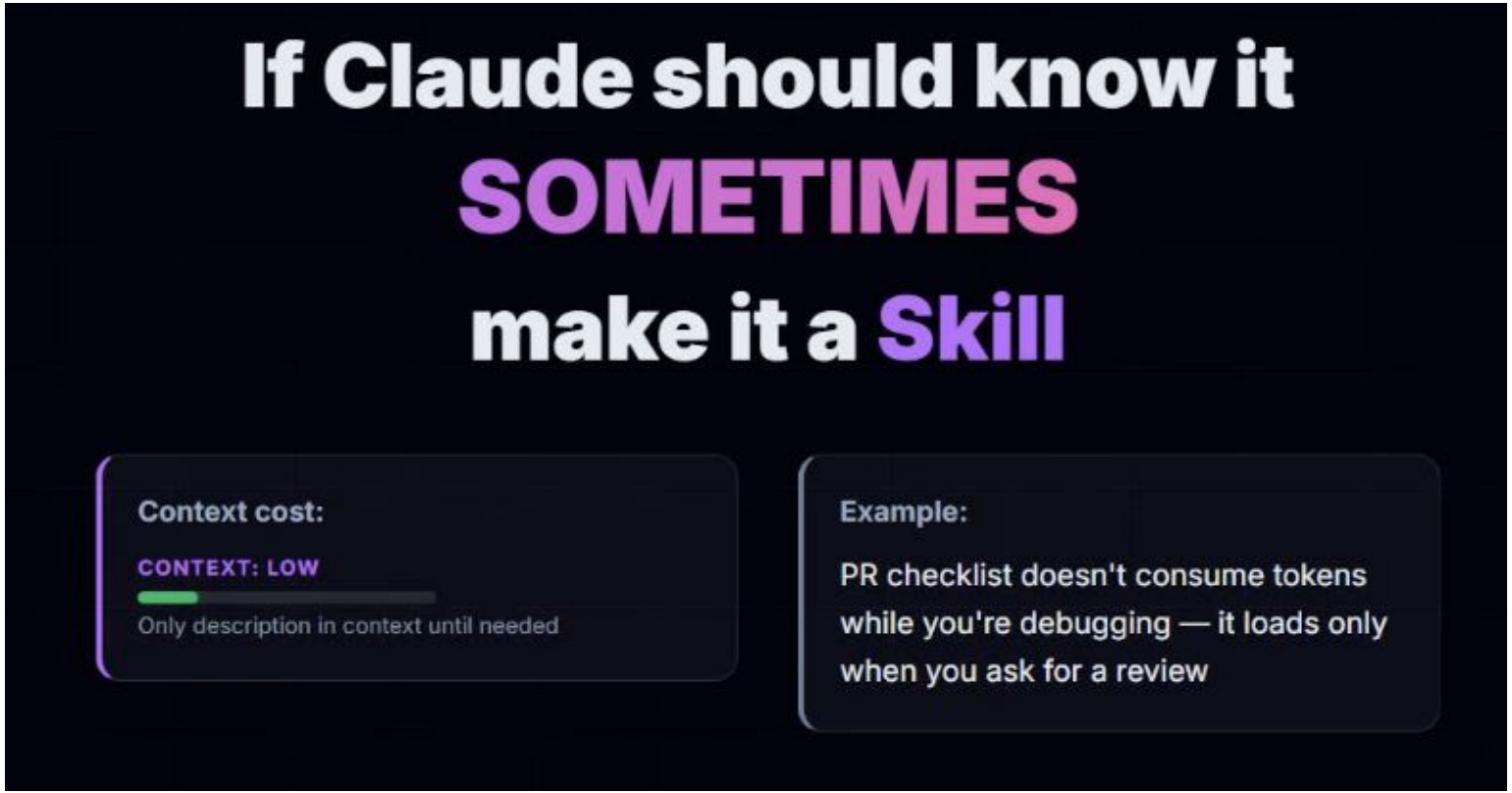
Key Rule - Claude.md

If Claude should **ALWAYS** know it →

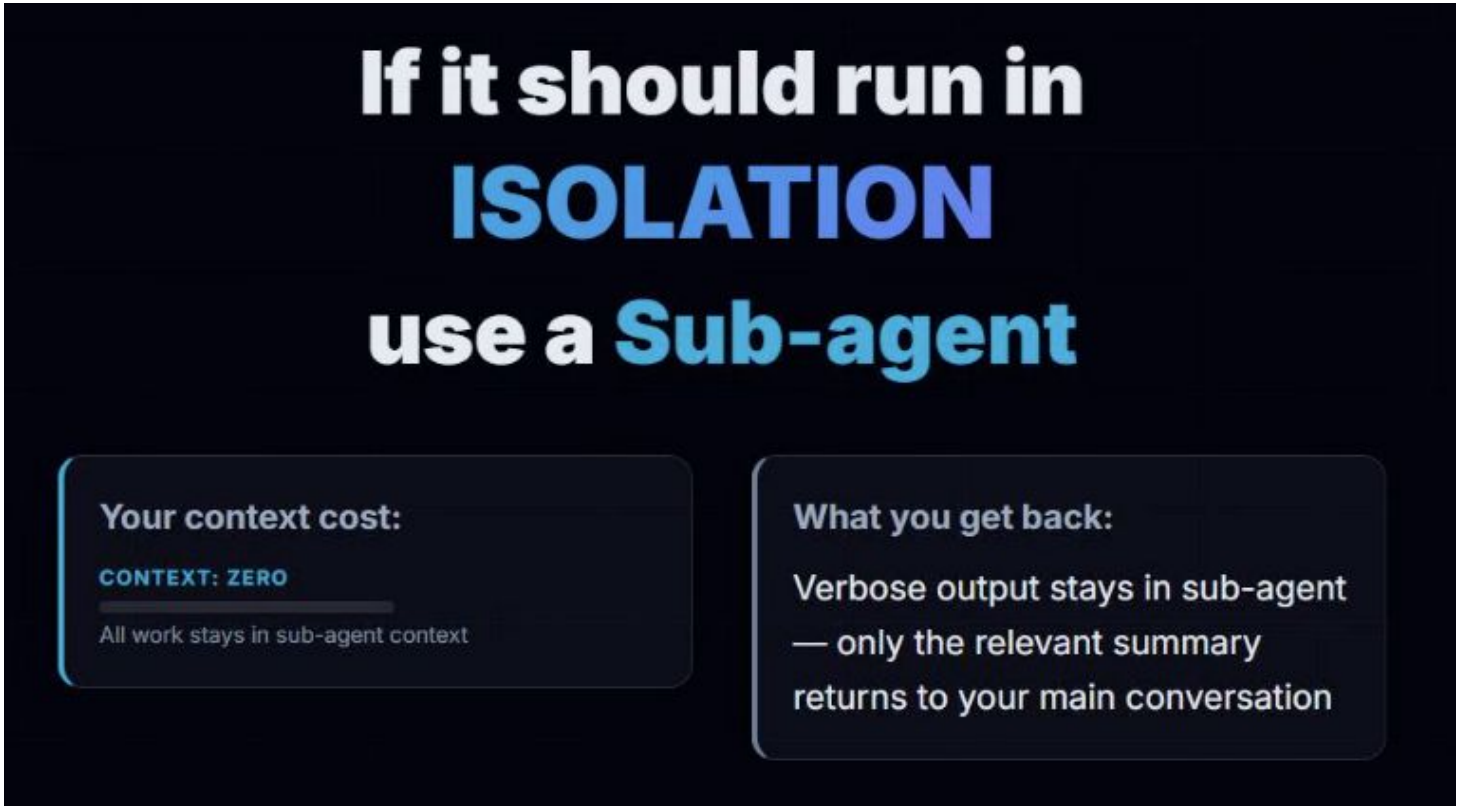
CLAUDE.md

⚠️ Everything in CLAUDE.md consumes your context window — every session, whether it's relevant or not.

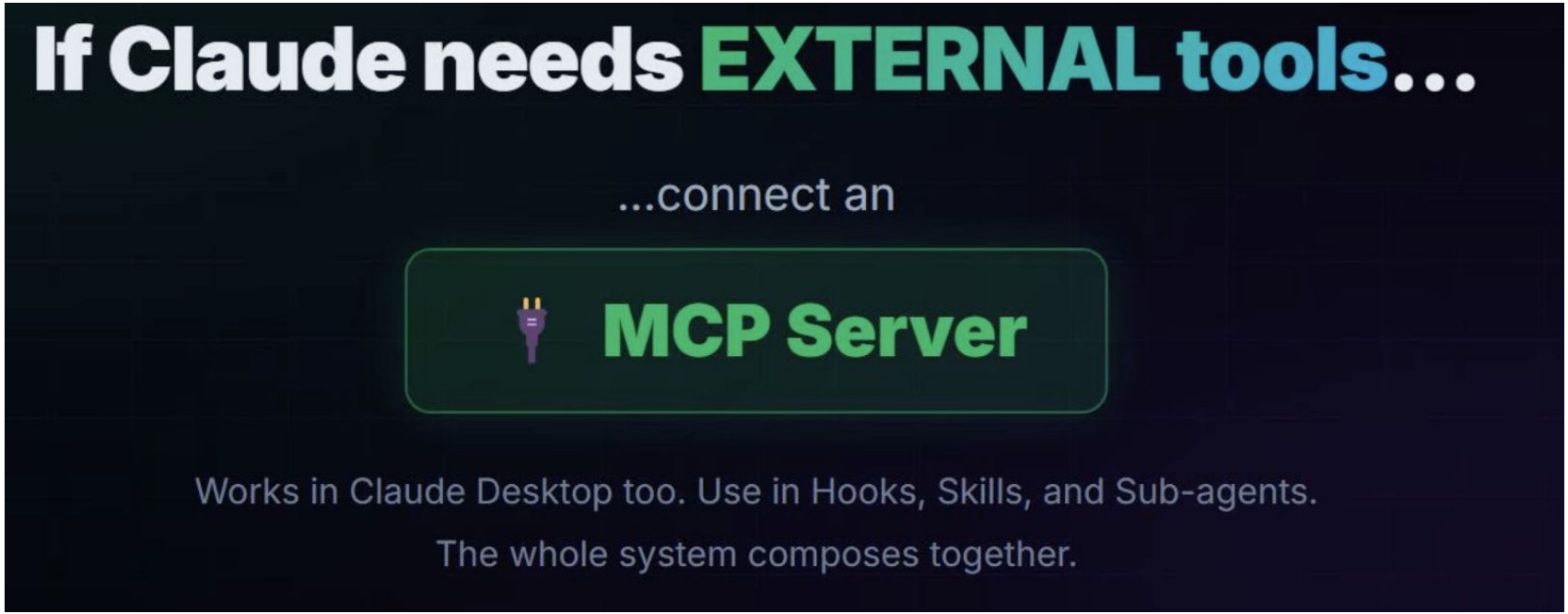
Key Rule - Skill



Key Rule - Sub-Agent



Key Role - MCP


If Claude needs EXTERNAL tools...
...connect an
MCP Server
Works in Claude Desktop too. Use in Hooks, Skills, and Sub-agents.
The whole system composes together.

Key Rule - Hooks

Key Rules for Customization

📋 Should Claude always know this?	CLAUDE.md
🎯 Should Claude know this sometimes?	Skills
🤖 Should this run in isolation?	Sub-agents
⚡ Should this happen automatically on events?	Hooks
🔌 Does Claude need external tools?	MCP Servers

How to start

1. **/init**
   Bootstrap your CLAUDE.md foundation
2. **Write a Skill**
   Next time you repeat yourself
3. **Add a Hook**
   Next time you want guardrails
4. **Create a Sub-agent**
   Next time you need isolation
5. **Connect an MCP Server**
   Next time you need external data

Get The Most Out of Your Claude Code Terminal

Configuration

euricom
TECH TRIBES

Settings file

Settings File Locations (Priority Order)

1. Enterprise

```
/Library/Application Support/ClaudeCode/managed-settings.json
C:\Program Files\ClaudeCode\managed-settings.json
```

See also
Claude.ai
Managed settings

2. Project Local (git ignored)

```
.claude/settings.local.json
```

3. Project Shared

```
.claude/settings.json
```

4. User Global

```
~/.claude/settings.json
```

Permissions Settings

```
"permissions": {
  "allow": [
    "Bash(npx:*)",
    "Edit",
    "Write",
  ],
  "deny": [
    "Bash(rm -rf:*)",
    "Read(./.env)",
    "Bash(sudo:*)",
  ],
  "ask": [
    "Bash(git commit:*)",
  ]
}
```

Set your permissions accurately and let the AI review it.

Sandbox Settings

Mac Only

Claude Code's `/sandbox` command enables a secure, isolated environment for running AI agent tasks in the terminal. It uses OS-level isolation (Filesystem & Network isolation) to let Claude execute and edit files autonomously within defined boundaries, reducing constant permission prompts

```
/sandbox
```

Config

```
"sandbox": {
  "enabled": true,
  "autoAllowBashIfSandboxed": true,
  "network": {
    "allowLocalBinding": true
  },
  ...
}
```

Plugins

Claude Code plugins are shareable, bundled packages for the Claude Code CLI that extend its functionality with custom slash commands

```
/plugin
/plugin marketplace add <url-or-path>
/plugin install <name>@<marketplace>

/plugin install frontend-design@anthropic
  > Install for you (user scope)
    Install for all collaborators on this repository (project scope)
    Install for you, in this repo only (local scope)
    Open homepage
    Back to plugin list
```

Get more out of Claude Code

euricom
TECH TRIBES

Worktrees

Claude Code integrates with Git worktrees to enable parallel, isolated development sessions, preventing file conflicts across multiple AI-assisted tasks

Start a named worktree session with the

```
claude --worktree feature-auth
```

Let the subagent run in a separated worktree (challenge mode)

```
---
name: Feature Builder
description: Builds isolated features
isolation: worktree
---
Your prompt here.
```

There are hooks (`WorktreeCreate`, `WorktreeRemove`) to hook up scripts

Tasks

Claude Code's tasks system is a built-in project planner and coordinator that stored on disk (e.g. under `~/.claude/tasks`)

When you give Claude a complex request (e.g. "implement feature X end-to-end"), the model can:

1. **Decompose**: use TaskCreate to break the goal into concrete subtasks.
2. **Link with dependencies**: mark which tasks depend on others, building a dependency graph so it knows which can run in parallel and which are blocked.
3. **Execute and update**: as work progresses, use *TaskUpdate* to move tasks through states and attach outputs or notes.
4. **Query current plan**: use *TaskList* / *TaskGet* to decide "what should I or a sub-agent do next?" without relying on fragile chat history.

Agent Team is also using the Task system.

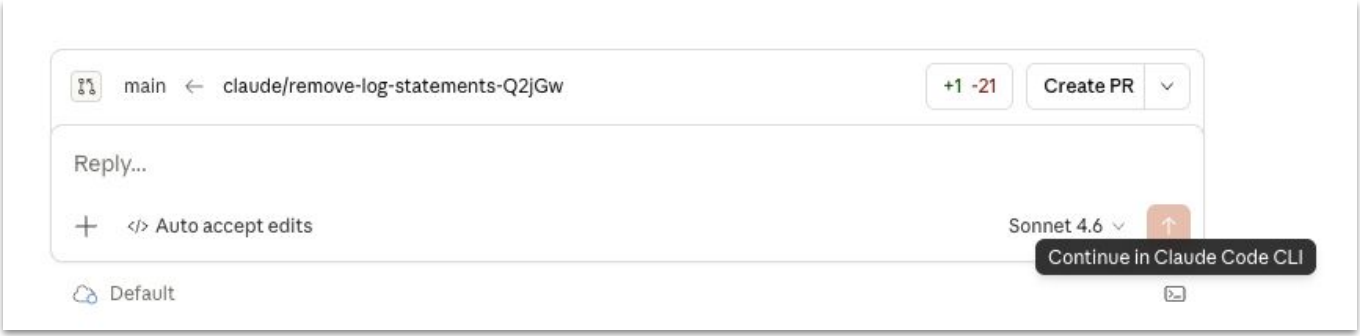
Preview Feature

Preview Feature

Claude Code on the web

Runs Claude Code from the browser in a secure cloud environment

Step 1: Open: https://claude.ai/code and run your session



Step 2: Open Claude Code in your terminal

```
claude --teleport session_012MhzYcjhy1puUJvpzR9yvA
```

Step 3: Then the complete session, with branch, context gets copied. And you can continue in Claude Code

Scheduling (Loop)

The `/loop` command runs a prompt or slash command on a recurring interval.
For example: `/loop 5m check the deploy` — this would check your deploy status every 5 minutes within the session.

```
/loop <interval> <prompt or command>
```

Examples:

- `/loop 10m run the test suite and summarize failures`
- `/loop 1m check if the server is still responding`
- `/loop 30s tail the logs and flag any errors`

List all cron jobs:

- `list your scheduled tasks`

Remove cron:

- `CronDelete <taskId>`

Scheduling (Loop)

	CLI / Loop	Desktop
Expires	3 days max	Never
Survives restart	No	Yes
Needs app open	Yes (terminal)	Yes
Creates new session	No, runs in current	Yes, new session created
Interrupts your work	Waits until you’re idle	Separate session

Statusline

Follow up usage, context, skills, ...

```
›

Model: Sonnet 4.6 | Ctx: 0 | ⎇main | (+179,-2) | Block: 2hr 35m                    ◐ medium · /effort
v2.1.72 | Skill: none                    Weekly Reset: 64hr 24m | Weekly: 22.0%
```

settings.json

```
{
  "statusLine": {
    "type": "command",
    "command": "npx -y ccstatusline@latest",
    "padding": 0
  }
}
```

config

```
# run from console
npx ccstatusline@latest
```

https://github.com/sirmalloc/ccstatusline

Improve Code

After a long conversation, an plan implementation and before you submit a PR

```
/simplify
```

Launch 3 agents, on your recent changes, automatic improves and fix issues.

- Code Reuse : Finds duplicated logic, utilities you forgot about
- Code Quality: Readability, naming, keep CLAUDE.md rules up to date.
- Efficiency: Unnecessary loops, missed concurrency

You can add a focus point

```
/simplify check for error handling gaps
```

Review

The /review and /security-review commands are slash commands within Claude Code for code quality checks and vulnerability scanning, respectively.

`/review` performs comprehensive code quality reviews, checking for code smells, anti-patterns, bugs, style consistency, error handling, unused code, and test coverage

```
/review
```

`/security-review` scans the codebase for vulnerabilities like SQL injection, XSS, authentication flaws, insecure data handling, and dependency issues

```
/security-review
```

Preview

From a terminal session, type `/desktop` to handover: Claude saves the session context and opens it in the **Claude Desktop** app (macOS/Windows), exiting the CLI afterward

```
/desktop
```

Config (.claude/launch.json)

```
{
  "configurations": [
    {
      "name": "dev",
      "runtimeExecutable": "bun",
      "runtimeArgs": ["run", "dev"],
      "port": 3000,
      "autoPort": true
    }
  ]
}
```

Preview - Multiple Servers

```
{
  "configurations": [{
      "name": "BE",  // Backend (e.g., DotNet API)
      "program": "dotnet",
      "args": ["run"],
      "port": 4000,  // BE listens here
      "cwd": "./backend"
    }, {
      "name": "FR",  // Frontend (e.g., Vite/React)
      "runtimeExecutable": "bun",
      "runtimeArgs": ["run", "dev"],
      "port": 3000,  // FE listens here
      "cwd": "./frontend",
      "env": {
        "VITE_API_URL": "http://localhost:4000",  // Pass BE port to FE via env
      }
    }]
}
```

https://code.claude.com/docs/en/desktop#configure-preview-servers

Insight

Is a built-in command that analyzes your recent Claude Code usage and generates a personal “coding-workflow report” about how you actually work with the tool.

```
/insights

Your shareable insights report is ready:
  file:///Users/<username>/.claude/usage-data/report.html

  Want to dig into any section or try one of the suggestions?
```

This command is globally and take all sessions for the last 30 days

https://code.claude.com/docs/en/desktop#configure-preview-servers

Batch

Enables parallel, large-scale code transformations across entire codebases using multiple AI agents. It automates migrations like framework swaps or naming convention updates, running up to 10x faster than individual prompts by processing independent work units simultaneously.

```
/batch
```

From PR

‘`claude --from-pr`’ is a convenience flag to jump straight into the session that’s associated with a specific GitHub pull request.

Start a session based on the PR

```
claude --from-pr
```

Remark: This only works when you initially create the PR via Claude Code (aka; gh pr create)

What we didn’t cover

- Commands
  - ‘`/voice`’
  - ‘`/chrome`’
  - ‘`/copy`’
  - ‘`/dokter`’
  - ‘`/install-github-app`’
- Agent Team
- Ascii Art in ‘Ask User Question’ tool
- Claude App
  - Ssh to remote server
  - Multi repo support
  - Git Diff tool with comments

https://www.youtube.com/watch?v=F_frSYyhzOE&t=72s

Securing Your Workspace

euricom
TECH TRIBES

What can go wrong

- **Vulnerable Code Generation**
  - No security review
- **Training Data Risks**
  - Your data/IP is used for training (free, Pro of Max plans)
- **Data and Secret Leaks**
  - .env
  - appsettings.json
  - ~/netrc
  - user.json
- **Prompt Injection Attacks**
  - Webfetch from malicious websites

Protection against security leaks & prompt injection

Claude Code

- Permissions
  - Bash, File & Network Access
- Hooks
  - Bash & File Access
  - Network Access
- Sandbox (Mac Only)
  - File & Network Access

3th Party

- Anthropic Hookify plugin
  - Bash & File Access
- Nono (prerelease)
  - Kernel-enforced sandbox for file & network access
- Varlock
  - Secure env vars
- Lasso Security
  - Prompt Injection

Closing Notes

euricom
TECH TRIBES

Claude Code is going fast

- +100 new features & improvements in the last months

- It's more an Ecosystem now
  - **Claude.ai** for research & corporate settings
  - **CoWork** for organization
  - **Claude Code CLI** for your implementation
  - **Claude App** for multi agents, preview, web agents....
  - **Mobile Claude App** for remote control

- After 1 year it's still the number 1 agentic development tool
- Using Claude Code can improve your AI Development journey